# Retrieval-Augmented Generation (RAG)

UNIST CSE402 Default Project

**Ho Mai Phuong**
Department of Computer Science & Engineering
UNIST
phuonggii@unist.ac.kr

## 1 Research paper summary (max 2 pages)

- **Title** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
- **Venue** Neural Information Processing Systems (Neurips)
- **Year** 2020
- **URL** https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

**Background.** Pre-trained neural language models function as parameterized implicit knowledge bases, learning a substantial amount of in-depth knowledge from data. However, the authors identify several critical problems with this approach:

- They cannot easily expand or revise their memory without costly training.
- They produce "hallucinations" - outputs that are fluent but factually incorrect.
- They can't straightforwardly provide insight into their predictions.

The paper combines pre-trained, parametric-memory generation models with a non-parametric memory through a general-purpose fine-tuning approach, which is refer to as retrieval-augmented generation (RAG). Specifically, the parametric memory is a pre-trained seq2seq transformer, and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever. By using pre-trained access mechanisms, the ability to access knowledge is present without additional training.

**Summary of contributions.** The paper introduces *Retrieval-Augmented Generation (RAG)* (Lewis et al., 2020), a hybrid architecture that combines a neural retriever with a generative component:

- The model uses a Dense Passage Retriever (DPR) to fetch relevant documents and a BART generator to produce the final output, with the entire system being fine-tuned end-to-end.
- The paper proposes **RAG-Sequence** (uses the same retrieved document to generate the entire sequence) and **RAG-Token** (allows the model to use different documents for each token in the generated output).
- The authors demonstrates that the model's "memory" could be updated by simply swapping the document index without retraining the model parameters, an update from models like T5 or BART.

RAG achieves good results on several open-domain question answering (QA) tasks, such as Natural Questions and WebQuestions, outperforming both purely parametric models and retrieval supervised systems.

**Limitations and discussion.** One limitation of the paper is its reliance on a fixed retrieval corpus (Wikipedia). While effective for general knowledge, Wikipedia is never entirely factual and completely devoid of bias. Also, the paper does not explore how the system behaves with rapidly changing data and the initial evaluation focused largely on English Wikipedia text. Despite these limitations, the paper is highly convincing because it provides a clear, modular solution to the hallucination problem

**Why this paper?** I chose this paper because it provides the foundation for my project, "Building GPT and RAG." Since my goal is to understand how to augment a generative model with external data, reading the paper that defines RAG is essential. I want to understand the mathematical background of how the retriever and generator interact during training. Having read it, I have gained a much clearer understanding of its architectures and its improvements compared to other models, which is a tremendous help for this NLP class project.

**Wider research context.** RAG represents a major shift from static systems to dynamic generate-from-retrieval systems. It connects to broad NLP themes in several ways:

- It suggests that language models should not try to memorize the world, but rather learn to read and reason over external data.
- It grounds the output in verifiable evidence, reducing hallucinations.
- RAG shows a variety in applications. It has been applied beyond QA to tasks like fact-checking

(FEVER), dialogue systems, showing that the retrieve-then-generate pattern is a robust architecture for many knowledge-intensive task.

## 2 Project description (1-2 pages)

**Goal.** The primary goal of this project is to implement a functional GPT-small architecture from scratch and integrate it into a Retrieval-Augmented Generation (RAG) pipeline. Aside from that, I plan to propose an extension where the retrieval index is not just "General Wikipedia," but a curated, domain-specific document store. I will test whether the GPT-small model, when fine-tuned on this specific domain, can achieve a higher *faithfulness* score than the baseline. If time permits, I will investigate whether Cross-Encoder reranking can reduce "hallucination" in open-domain Question Answering compared to standard Bi-Encoder retrieval.

This project implements the "retrieve-then-generate" framework in my chosen paper but adapts both a general-purpose system and a revised system. While the original paper used a large BART model and English Wikipedia, I am testing if the same architecture remains effective when using a GPT-small model paired with a specialized technical index. I also want to confirm that the RAG framework is efficient to reduce hallucinations in "long-tail" topics that general models typically struggle with.

**Task.** The project addresses two main tasks:

1. Language Modeling Analysis: Pretraining GPT-small and finetuning.
2. Building RAG system and improving the baseline system by proposing novel ideas.

**Data.**

- Pretraining Data: I intend to use a subset of the OpenWebText or TinyStories dataset (approx. 100MB–500MB) to train the GPT-small model from scratch.
- QA Data: The Natural Questions (NQ) dataset. I will use the standard split provided in the default project files (roughly 100k training pairs).
- Knowledge Base: A processed Wikipedia snippet collection (DPR-indexed) provided in the project skeleton.

**Methods.** I will implement the following:

- Transformer: I will complete missing codes for parts of Multi-Head Self-Attention (MHSA) with Grouped Query Attention (GQA), RoPE (Rotary Positional Embeddings), and RMSNorm.
- RAG Pipeline: A system using DPR for initial document retrieval.
- Extension method: I will read papers improving the idea of RAG and apply them to create a novel idea. I have been thinking about Domain-Specific RAG or Cross-Encoder reranker, which performs full self-attention over the query for the top $k$ results to select the best context for the generator.

**Baselines.** I will compare my revised system against the following baselines:

- GPT-small (No RAG): The model generating answers based purely on its internal parameters.
- Vanilla RAG Baseline: The provided skeleton code using Llama-3.2-1B-Instruct with top-k retrieval, as defined in the project instructions.

**Evaluation.** I will use the following evaluation metrics:

- Report Exact Match (EM) scores: check if the generated answer matches the ground truth exactly.
- Perplexity: evaluate the pretraining quality of the GPT-small model.

I will compare my results against the performance scores reported in the above RAG paper and the benchmarks provided in the course handout. I expect the extensions to improve the EM score by at least 5% over the vanilla RAG baseline.

## References

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.